

Contenido

Variables y Tipos de datos	3
Resumen de tipos de variables	3
Lectura de datos en Python	3
Números.....	4
Cadenas de texto (string).....	4
Concatenación de strings.....	5
Conversión de tipos.....	6
Métodos en cadenas de texto (string)	6
Operadores y expresiones.....	7
Operadores aritméticos.....	7
Operadores relacionales o de comparación	7
Operadores lógicos.....	8
Estructuras de control.....	8
Condicionales.....	8
Sentencia IFif..else	8
Bucles.....	9
Sentencia WHILE	9
Sentencia FOR	10
Listas y tuplas	12
Tuplas.....	12
Diccionarios	13
Funciones	14
Definir y llamar a una función.....	14
Funciones con argumentos múltiples.....	15
Ámbito de las variables (scope).....	15
Excepciones	15
Clases y objetos.....	16
Atributos de clase vs Atributos de instancia	17
Private y protected.....	18
Herencia	18
Herencia múltiple	19
Módulos y Paquetes	19
Módulos	19

Localización de los módulos20

Paquetes20

Variables y Tipos de datos

Las variables permiten almacenar datos del programa. Las variables serán de un tipo u otro en función de la información que se guarde en ellas. El nombre de una variable se conoce como **identificador**, y deberá cumplir las siguientes reglas:

- Comenzar con una letra o un guión bajo
- El resto de nombre estará formado por letras, números o guiones bajos.
- Los nombres de las variables son case sensitive, es decir, no es lo mismo que una variable se llame `resultado` que `RESULTADO`.
- Existe una serie de palabras reservadas que no se pueden utilizar (`def`, `global`, `return`, `if`, `for`, ...).

Existen algunas recomendaciones respecto a los nombres de las variables, recogidas en la [guía de estilo PEP8 de Python](#):

Utilizar nombres descriptivos, en minúsculas y separados por guiones bajos si fuese necesario:
`resultado`, `mi_variable`, `valor_anterior`,...

Escribir las constantes en mayúsculas: `MI_CONSTANTE`, `NUMERO_PI`, ...

Antes y después del signo `=`, debe haber uno (y solo un) espacio en blanco.

Resumen de tipos de variables

```
edad = 24 # número entero (integer)
precio = 112.9 # número de punto flotante (float)
titulo = 'Aprende Python desde cero' # cadena de texto (string)
test = True # booleano
```

Lectura de datos en Python

La función `input()` permite introducir datos al usuario:

```
>>> nombre = input()
Leire
>>> print(nombre)
Leire
```

Como se puede ver en el siguiente ejemplo, es posible pasarle como argumento el mensaje a mostrar al usuario.

```
>>> nombre = input("Escribe tu nombre: ")
Escribe tu nombre: Leire
>>> print(nombre)
Leire
```

Números

Python soporta dos tipos de números: enteros (integer) y de punto flotante (float)

```
# integer
x = 5
print(x)

# float
y = 5.0
print(y)

# Otra forma de declarar un float
z = float(5)
print(z)
```

Si tenemos dudas del valor de una variable, podemos utilizar la función `type()`:

```
>>> x = 5.5
>>> type(x)
<class 'float'>
```

Cadenas de texto (string)

Las cadenas de texto o strings se definen mediante comilla simple (' ') o doble comilla (" "):

```
mi_nombre = 'Ane'
print(mi_nombre)
mi_nombre= "Ane"
print(mi_nombre)
```

La diferencia principal es la mayor facilidad de las comillas dobles para introducir apóstrofes:

```
mi_nombre = 'I\'m John'
print(mi_nombre)
mi_nombre= "I'm John"
print(mi_nombre)
```

Más información sobre strings y caracteres especiales en:

<https://docs.python.org/3/tutorial/introduction.html#strings>

Para definir strings multi-línea se utiliza la triples comillas ("""):

```
frase = """ esto es una
frase muy larga de más de
una línea ..."""
```

Concatenación de strings

Es posible unir dos strings con el operador +:

```
>>> primera_palabra = 'Hola'
>>> frase_completa = primera_palabra + ', mundo'
>>> print(frase_completa)
'Hola, mundo'

>>> segunda_palabra = 'mundo'
>>> frase_completa = primera_palabra + ', ' + segunda_palabra
>>> print(frase_completa)
'Hola, mundo'
```

Método alternativo 1: `str.join()`: El método `join()` recibe como argumento el listado (de tipo List, Tuple, String, Dictionary y Set) de strings que se desean concatenar. Se invoca sobre el separador (el cual a su vez es un string también):

```
>>> strings = ['do', 're', 'mi']
>>> separador = ' - '
>>> separador.join(strings)
'do - re - mi'
```

Para iterar un elemento detrás del otro se introducirá como separador un string vacío:

```
>>> strings = ['do', 're', 'mi']
>>> ''.join(strings)
'doremi'
```

Método alternativo 2: `str.format()`: Python 3 introdujo una nueva forma para formatear strings, la cual sustituye a la anterior en la que se hace uso del operador `%`. Para ello se invoca el método `format()` de un string:

```
# Ordenado por defecto:
frase = "Meses: {}, {} y {}".format('Enero', 'Febrero', 'Marzo')
print(frase)

# Especificar el orden indicando la posición:
frase = "Meses: {1}, {0} y {2}".format('Enero', 'Febrero', 'Marzo')
print(frase)

# Especificar el orden mediante parejas clave-valor:
orden_palabra = "Meses: {ene}, {feb} y {mar}".format(ene='Enero',
feb='Febrero', mar='Marzo')
print(frase)
```

Conversión de tipos

A la hora de concatenar un string con otras variables como integer o float puede haber problemas:

```
>>> edad = 25
>>> nota_media = 7.3
>>> print("Tengo " + edad + " años y mi nota media es " + nota_media
+ ".")
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Mediante la función `str()` podemos convertir un valor a string:

```
>>> edad = 25
>>> nota_media = 7.3
>>> print("Tengo " + str(edad) + " años y mi nota media es " +
str(nota_media) + ".")
Tengo 25 años y mi nota media es 7.3.
```

De igual manera es posible convertir a otros tipos con las funciones `int()`, `float()` and `bool()`.

Métodos en cadenas de texto (string)

Es posible **obtener un carácter concreto** de un string utilizando los corchetes `[]`:

```
frase = 'Aprendiendo a programar en Python'
frase[0] # devuelve el primer caracter
frase[1] # devuelve el segundo caracter
frase[-1] # devuelve el primer caracter empezando por el final
frase[-2] # # devuelve el segundo caracter empezando por el final
```

Si queremos **obtener un substring**, utilizaremos la siguiente notación:

```
frase = 'Aprendiendo a programar en Python'
mi_substring = frase[1:5] # devuelve los caracteres desde la posición
1 hasta la 5 (no incluye el 5)
```

En caso de dejar la primera variable vacía, se considera la primera posición del string. Dejando la segunda variable vacía se considera la última posición del string:

```
>>> frase = 'Aprendiendo a programar en Python'
>>> mi_substring = frase[:5]
>>> mi_substring
'Apren'
>>> mi_substring = frase[4:]
>>> mi_substring
'ndiendo a programar en Python'
```

Otros métodos útiles de string:

- `endswith`: saber si una cadena termina con
- `startswith`: saber si una cadena empieza con
- `isalpha`: saber si una cadena contiene letras únicamente
- `isalnum`: saber si una cadena contiene números y letras únicamente (alfanumérico)
- `isdigit`: saber si una cadena contiene sólo dígitos.
- `isnumeric()`: devuelve True si `str` contiene solamente números
- `isspace`: saber si una cadena contiene sólo espacios.
- `istitle`: saber si una cadena es un título
- `islower`: saber si una cadena contiene todos sus caracteres en minúsculas
- `isupper`: saber si una cadena contiene todos sus caracteres en mayúsculas
- `upper` : convierte a mayúsculas
- `lower`: convierte a minúsculas
- `title`: convierte a mayúsculas la primera letra de cada palabra
- `count()`
- `find('d')`: devuelve el índice de la primera aparición de 'd' (o -1 si no lo encuentra)
- `replace('p', 'q')`
- `substr in str`: devuelve True si el string contiene el substring
- `replace(old, new [, count])`: reemplaza 'old' por 'new' un máximo de 'count' veces.

Operadores y expresiones

Los operadores son símbolos especiales que permiten realizar operaciones aritméticas o lógicas.

Operadores aritméticos

Los operadores aritméticos se utilizan para realizar operaciones matemáticas (suma, resta, multiplicación,...):

Operador	Ejemplo	Significado
+	<code>a + b</code>	Suma
-	<code>a - b</code>	Resta
-	<code>-a</code>	Negación (asignar valor negativo)
*	<code>a * b</code>	Multiplicación
/	<code>a / b</code>	División
%	<code>a % b</code>	Módulo (resto de la división)
//	<code>a // b</code>	División entera
**	<code>a ** b</code>	Exponente

Operadores relacionales o de comparación

Los operadores relacionales se utilizan para comparar valores y devuelven como resultado un booleano: True o False.

Operador	Ejemplo	Significado
>	a > b	Mayor que: True si a es mayor que b
<	a < b	Menor que: True si a es menor que b
==	a == b	Igual: True si a y b son iguales
!=	a != b	Distinto: True si a y b son distintos
>=	a >= b	Mayor o igual: True si a es igual o mayor que b
<=	a <= b	Menor o igual: True si a es igual o menor que b

Operadores lógicos

Los operadores lógicos `and`, `or` y `not`.

Operador	Ejemplo	Significado
and	a and b	True si a y b son True
or	a or b	True si a o b son true
not	not b	True si b es falso

Estructuras de control

Condicionales

Las estructuras de control se utilizan para **ejecutar bloques de código en función de condiciones**.

Sentencia IFif..else

Si se cumple la condición especificada en el `if`, se ejecutará el bloque indentado. En caso de que el resultado de la condición sea `False`, no se ejecutará:

```
numero = 5
if numero > 1:
    print("Es mayor que uno")
```

Es posible especificar un bloque de código que se ejecute en caso de que la condición no se cumpla:

```
numero = 2
if numero > 10:
    print("Es mayor que diez")
else:
    print("Es menor o igual que diez")
```


Mediante la expresión `elif` podemos comprobar otras condiciones. Se seguirán comprobando todos los `elif` hasta que uno cumpla la condición especificada. En caso contrario, se ejecutará el bloque de código dentro de `else` (si lo hubiera)

```
numero = 5
if numero < 3:
    print("Es menor que 3")
elif numero < 6:
    print("El número está entre el 3 y el 5")
else:
    print("Es mayor o igual a 6")
```

Otra opción es anidar distintos bloques de `ifs`.

```
numero = 2
if numero >= 0:
    if numero == 0:
        print("El valor es 0")
    else:
        print("Es un número positivo")
else:
    print("Es un número negativo")
```

Bucles

Los bucles permiten ejecutar un bloque de código tantas veces como queramos.

Sentencia WHILE

La sentencia `while` permite ejecutar un bloque de código mientras la expresión que definamos se cumpla (es decir, devuelva `True`). Python interpretará como `True` cualquier valor distinto a `0` o `None`.

```
contador = 0
while(contador < 5):
    contador = contador+1
    print("Iteración número {}".format(contador))
print ("¡Fin!")
```

Para detener una ejecución se utiliza la sentencia `break`:

```
contador = 0
while(contador < 5):
    contador = contador+1
    print("Iteración número {}".format(contador))
    if contador == 3:
        break
print ("¡Fin!")
```

También es posible saltar únicamente la iteración actual mediante la sentencia `continue`:

```
contador = 0
while(contador < 5):
    contador = contador+1
    if contador == 3:
        continue
    print("Iteración número {}".format(contador))
print ("¡Fin!")
```

La salida del programa anterior será la siguiente:

```
Iteración número 1
Iteración número 2
Iteración número 4
Iteración número 5
Bucle while finalizado
```

Bucle WHILE con ELSE

Para ejecutar un código al finalizar, es decir, cuando la condición es `False` y no se ha finalizado con `break`, se utiliza la expresión `else`:

```
count = 0
while(count < 5):
    count = count+1
    print("Iteración número {}".format(count))
else:
    print ("Bucle while finalizado")
```

Sentencia FOR

A diferencia de otros lenguajes de programación, en Python la sentencia `FOR` itera únicamente por secuencias (listas, tuplas, cadenas de caracteres,...).

```
alumnos = ["Ane", "Mikel", "Unai", "Lorea"]
for alumno in alumnos:
    print(alumno)
```

También es posible utilizarlo para recorrer un string:

```
frase = "Aprendiendo Python"
for c in frase:
    print(c)
```

Para detener una ejecución se utiliza la sentencia `break`:

```
numeros = [4,8,2,7,1,9,3,5]
total = 0
```

```

for n in numeros:
    total += n
    if total > 10:
        break
print(total)

```

También es posible saltar únicamente la iteración actual mediante la sentencia `continue`:

```

numeros = [4,8,2,7,1,9,3,5]
total = 0

# solo sumar los números impares
for num in numeros:
    if num % 2 == 0:
        print("Numero par, no lo sumamos")
        continue
    total += num

```

Bucle FOR con ELSE

Python permite definir un bloque de código a ejecutar cuando la iteración por los elementos de una lista finaliza. Es importante mencionar que no se ejecutará si se ha finalizado mediante `break`.

```

alumnos = ["Ane", "Mikel", "Unai", "Lorea"]
for alumno in alumnos:
    print(alumno)
else:
    print("No quedan más alumnos.")

```

La función range()

La función `range([start,] stop [, step])` devuelve una secuencia de números. Es por ello que se utiliza de forma frecuente para iterar:

```

for i in range(3):
    print(i)
# 0
# 1
# 2

```

También podemos indicar el inicio, fin y step:

```

print("Números del 5 al 10")
for i in range(5, 10):
    print(i, end=', ')
# 5, 6, 7, 8, 9,

print("Números impares del 1 al 10")
for i in range(1, 10, 2):
    print(i, end=', ')
# 1, 3, 5, 7, 9,

```

Para iterar por una lista utilizando el índice, basta con combinarlo con la función `len()`:

```
alumnos = ["Ane", "Mikel", "Unai", "Lorea"]
for i in range(len(alumnos)):
    print(alumnos[i])
```

Listas y tuplas

Las listas permiten **guardar más de un elemento** dentro de una variable en un orden concreto. Pueden contener un número **ilimitado** de variables de **cualquier tipo**:

```
alumnos = ["Ane", "Unai", "Itziar", "Mikel"]
print(alumnos[0]) # muestra "Ane"
print(alumnos[1]) # muestra "Unai"
print(alumnos[2]) # muestra "Itziar"
print(alumnos[-1]) # muestra "Mikel"
```

Los **métodos más utilizados** con las listas son los siguientes:

Método	Acción
<code>alumnos.append("Amaia")</code>	Inserta "Jon" al final de la lista
<code>alumnos.insert(0, "Amaia")</code>	Inserta "Amaia" en la posición 0
<code>alumnos.remove("Amaia")</code>	Elimina la primera aparición de "Amaia" de la lista
<code>alumnos.pop()</code>	Elimina el último elemento de la lista
<code>alumnos.clear()</code>	Elimina todos los elementos de la lista
<code>alumnos.index("Amaia")</code>	Devuelve el índice de la primera aparición de "Amaia"
<code>alumnos.sort()</code>	Ordena la lista (los elementos deben ser comparables)
<code>sorted(alumnos)</code>	Devuelve una copia de la lista 'alumnos' ordenada (no ordena la pasada como parámetro)
<code>alumnos.reverse()</code>	Ordena la lista en orden inverso
<code>alumnos.copy()</code>	Devuelve una copia de la lista
<code>alumnos.extend(otra_lista)</code>	Fusiona las dos listas

Tuplas

Las tuplas son **listas inmutables**. Es decir, una vez declaradas, no se pueden realizar modificaciones sobre ellas (añadir/eliminar elementos o hacer modificaciones sobre ellos). Para definir una tupla se escriben los elementos entre paréntesis:

```
valores = (1,2,3)
```

El acceso a sus elementos se hace de igual que con listas:

```
valores = ("a", "b", "c", "d", "e", "f")
print(valores[1])
print(valores[2:4])
```

Una acción típica de las tuplas es "desempaquetar" (unpack) sus valores:

```
a, b, c = valores
```

Diccionarios

Un diccionario es un conjunto de parejas **clave- valor** (key-value). Es decir, se accede a cada elemento a partir de su clave. Se definen de la siguiente manera:

```
estudiante = {  
    "nombre": "Iñaki Perurena",  
    "edad": 30,  
    "nota_media": 7.25,  
    "repetidor" : False  
}
```

Las **claves tienen que ser únicas** y estar formadas por un **string o un número**. Para acceder al valor de una clave existen dos maneras distintas:

```
edad = estudiante["edad"] # devuelve el valor de 'edad'  
nota_media = estudiante.get("nota_media") # devuelve el valor de  
'nota_media'  
  
estudiante["edad"] = 25 # actualiza el valor de 'edad'  
estudiante["suspensos"] = 3 # inserta una nueva pareja clave - valor  
estudiante.update({'aprobados':'8'}): inserta una nueva pareja clave  
- valor o actualiza su valor si ya existiera
```

Algunos de los métodos más utilizados son los siguientes:

Método	Acción
diccionario.keys()	Devuelve todas las claves del diccionario
diccionario.values()	Devuelve todos los valores del diccionario
diccionario.pop(clave[, <default>])	Elimina la clave del diccionario y devuelve su valor asociado. Si no la encuentra y se indica un valor por defecto, devuelve el valor por defecto indicado.
diccionario.clear()	Vacía el diccionario
diccionario.clear()	Vacía el diccionario
clave in diccionario	Devuelve True si el diccionario contiene la clave o False en caso contrario.
valor in diccionario.values()	Devuelve True si el diccionario contiene el valor o False en caso contrario.

Funciones

Una función es un grupo de sentencias que realizan una tarea concreta. Esta forma de agrupar código es una forma de **ordenar** nuestra aplicación en pequeños bloques, **facilitando así su lectura** y permitiendo **reutilizar** el código que contienen sin esfuerzo.

Definir y llamar a una función

La sintaxis de una función en Python es la siguiente:

```
def saludo(nombre):  
    # código de la función  
    print("Hola, " + nombre+ ". ¡Bienvenido!")
```

Se escribe la palabra reservada `def` seguida del nombre de la función y sus parámetros entre paréntesis.

Para **llamar a una función** solo hay que escribir el nombre de la función seguida de los parámetros (si los hubiera) entre paréntesis.

```
>>> saludo('Maitane')  
Hola, Maitane. ¡Bienvenida!
```

Es posible asignar al parámetro un **valor por defecto**.

```
def saludo(nombre = "Anónimo"):  
    print("Hola, " + nombre+ ". ¡Bienvenido!")  
  
saludo("Leire") # Hola, Maitane. ¡Bienvenida!  
saludo() # Hola, Anónimo. ¡Bienvenida!
```

Existen dos tipos de parámetros o argumentos:

- **Parámetros posicionales:** la posición en la que se pasan importa
- **Parámetros con palabra clave (keyword arguments):** la posición no importa, se indica una clave para cada parámetro.

```
def suma(a, b):  
    resultado = a + b  
    print(resultado)  
suma(45, 20) # parámetros posicionales  
suma(a = 45, b =20) # parámetros mediante clave
```

Las funciones pueden **devolver un valor** utilizando la palabra `return`. Una vez devuelto un valor, la función finaliza su ejecución.

```
def suma(a, b):  
    resultado = a + b  
    return resultado
```

```
print(suma(4,5)) # 9
```

Funciones con argumentos múltiples

Es posible recibir un **número desconocido** de parámetros añadiendo un `*` en la definición de la función.

```
def suma_todo(*args):
    resultado = 0
    for i in args:
        resultado += i
    return resultado
v, w, x, y, z = 5, 2, 12, 6, 9
total = suma_todo(v, w, x, y, z)
print("La suma total es:" + str(total)) # La suma total es: 34
```

Ámbito de las variables (scope)

El ámbito de una variable (*scope*) se refiere a la zona del programa dónde una variable "existe". Fuera del ámbito de una variable no podremos acceder a su valor ni manejarla.

Los parámetros y variables definidos en una función no estarán accesibles fuera de la función. A este ámbito se le conoce como **ámbito local**. Es importante mencionar que una vez ejecutada una función, el valor de las variables locales no se almacena, por lo que la próxima vez que se llame a la función, ésta no recordará ningún valor de llamadas anteriores.

```
def calcula():
    a = 1
    print("Dentro de la función:", a)

a = 5
calcula()
print("Fuera de la función:", a)

### Output ###
# Dentro de la función:1
# Fuera de la función:5
```

Por el contrario, las variables definidas fuera de una función sí que están accesibles desde dentro de la función. Se considera que están en el **ámbito global**. No obstante, no se podrán modificar dentro de la función a no ser que estén definidas con la palabra clave `global`.

Excepciones

Las excepciones son **errores en la ejecución de un programa** que hacen que el programa termine de forma inesperada. Normalmente ocurren debido a un uso indebido de los datos

(p.ej. una división entre cero). La manera de controlar las excepciones es agrupando el código en 2 bloques (más 1 opcional):

- `try`: agrupa el bloque de código en el que se pueda dar una excepción.
- `catch`: contiene el código a ejecutar en caso de que la excepción haya sido lanzada.
- `finally` (opcional): permite ejecutar un bloque de código siempre, se haya capturado o no una excepción.

```
try:
    numero = int(input('Introduce un número: '))
    dividendo = 150
    resultado = dividendo / numero
    print(resultado)
except ValueError:
    print('Número inválido')
except ZeroDivisionError:
    print('No se puede dividir entre 0')
finally:
    print("Ejecutando finally antes de salir")
```

También es posible lanzar excepciones de forma controlada mediante la sentencia `raise`.

```
raise NameError(';Soy una excepción!')
```

Clases y objetos

Python soporta la programación orientada a objetos. Esto quiere decir que podemos definir entidades agrupando (encapsulando) sus atributos y comportamiento (métodos) en clases.

La definición de una clase en Python se hace de la siguiente manera:

```
class Persona:
    # atributos
    nombre = "Josune"
    edad = 24

    # metodos
    def camina(self):
        print(self.nombre + " está caminando")
```

A partir de una clase se pueden crear tantas **objetos** como se desee. Los objetos de una clase se conocen como **instancias**. Cada objeto **contiene los atributos y métodos de la clase** y podrá asignar a esos atributos unos valores concretos. Esto se conoce como el **estado de un objeto**.

```
p1 = Persona() # la variable p1 contiene un objeto de la clase
Persona
p1.camina()
print(p1.nombre)
print(p1.edad)
```


Una función dentro de una clase se conoce como **método**. Las clases contienen el método especial `__init__` conocido como **constructor** y que sirve para inicializar un objeto. Al crear un objeto siempre se llama al constructor. Una diferencia importante con otros lenguajes como Java es que solo se puede definir un único constructor.

```
class Persona:
    def __init__(self, nombre, apellidos, edad):
        self.nombre= nombre
        self.apellidos = apellidos
        self.edad = edad

    def camina(self):
        print(self.nombre + " está caminando")
```

En la creación del objeto es necesario indicar los argumentos del constructor:

```
p1 = Persona("Mike", "Mendiola", 25)
p1.camina()
print(p1.nombre)
print(p1.edad)
```

El parámetro `self` de los métodos es una referencia a la propia instancia y se utiliza para acceder a las variables que pertenecen a la clase. Si no se define un constructor, la clase hereda uno que únicamente recibe el argumento `self`.

Atributos de clase vs Atributos de instancia

Los atributos definidos dentro del constructor se conocen como **atributos de instancia**, por lo tanto, los atributos definidos dentro de la clase pero fuera del constructor se conocen como **atributos de clase**.

La principal diferencia es que un atributo de clase puede ser accedido aunque no existan instancias de la clase. Además, si se modifica su valor, se modificará el valor en todas las instancias existentes de dicha clase.

```
class Demo:
    atrib_estatico = 123 # compartido por todos los objetos
    def __init__(self, numero):
        self.atrib_instancia = numero # específico de cada objeto

c1 = Demo(456)
c2 = Demo(789)

# Valor inicial
print("C1: Estatico {0} - Instancia: {1}".format(c1.atrib_estatico,
c1.atrib_instancia))
# output: C1: Estatico 123 - Instancia: 456
print("C2: Estatico {0} - Instancia: {1}".format(c2.atrib_estatico,
c2.atrib_instancia))
# output: C2: Estatico 123 - Instancia: 789
```

```

Demo.atrib_estatico = -1

print("C2: Estatico {0} - Instancia: {1}".format(c2.atrib_estatico,
c2.atrib_instancia))
# output: C2: Estatico -1 - Instancia: 456
print("C2: Estatico {0} - Instancia: {1}".format(c2.atrib_estatico,
c2.atrib_instancia))
# output: C2: Estatico -1 - Instancia: 789

c1.atrib_instancia = 999

print("C1: Estatico {0} - Instancia: {1}".format(c1.atrib_estatico,
c1.atrib_instancia))
# output: C1: Estatico -1 - Instancia: 999

print("C2: Estatico {0} - Instancia: {1}".format(c2.atrib_estatico,
c2.atrib_instancia))
# output: C2: Estatico -1 - Instancia: 789

```

Private y protected

A diferencia de otros lenguajes de Programación Orientada a Objetos, todos los métodos y atributos en Python son públicos. Es decir, **no es posible definir una variable como private o protected**.

Existe una convención de añadir como prefijo un **guión bajo** (`_`) a los atributos que consideramos como **protected** y dos guiones bajos (`__`) a las variables que consideramos private.

```

class Persona:
    def __init__(self, nombre, edad):
        self._nombre = nombre # atributo protected
        self.__edad = edad # atributo private

```

Herencia

La herencia es una técnica de la Programación Orientada a Objetos en la que una clase (conocida como **clase hija** o **subclase**) hereda todos los métodos y propiedades de otra clase (conocida como **padre** o **clase base**).

La sintaxis para definir una clase que herede de otra es la siguiente:

```

class ClaseBase:
    # código de la clase base

class Subclase(BaseClass):
    # código de la subclase

```

La subclase puede añadir funcionalidades. Esta técnica permite **reutilizar código**.

```

class Dispositivo:

```

```

def __init__(self, identificador, marca):
    self.identificador = identificador
    self.marca = marca

def conectar(self):
    print(";Conectado!")

# la clase base se indica entre paréntesis
class Teclado(Dispositivo):
    def __init__(self, identificador, marca, tipo):
        # llamada al constructor del padre
        Dispositivo.__init__(self, identificador, marca)
        self.marca = marca
    # metodo de la subclase
    def pulsar_tecla(self, tecla):
        print(tecla)

t1 = Teclado("0001", "Logitech", "AZERTY")
t1.conectar()
t1.pulsar_tecla("a")

```

Herencia múltiple

Python soporta la herencia múltiple, es decir, heredad métodos y atributos de más de un padre. En caso de heredar atributos o métodos con el mismo nombre, Python dará prioridad al posicionado más a la izquierda en la declaración.

```

# en caso de conflicto Dispositivo tendrá prioridad sobre Periférico
class Teclado(Dispositivo, Periférico):
    # cuerpo de la clase

```

Módulos y Paquetes

Módulos

Un módulo es un archivo de Python que contiene variables, funciones y clases. Es una forma de ordenar y reutilizar código ya que todo el contenido de un módulo es accesible por los archivos que lo importen.

```

# mundo.py

def hola_mundo():
    print(";Hola Mundo!")

def adios_mundo():
    print(";Adios Mundo!")

```

Para acceder a las funciones desde otro archivo Python se utiliza la palabra reservada `import`:

```

# app.py

```

```
import mundo

# Llamada a la función
mundo.hola_mundo()
```

También existe la posibilidad de importar únicamente objetos concretos de un módulo mediante la sintaxis `from ... import:`

```
# app.py

from mundo import adios_mundo

# Llamada a la función
adios_mundo()
```

De esta forma no es necesario escribir el nombre del módulo antes de utilizar la función. De igual manera, se pueden importar varios objetos de un módulo separándolos por una coma:

```
# app.py

from mundo import adios_mundo, hola_mundo
```

Para importar todos los los objetos de un módulo basta con utilizar el asterisco:

```
# app.py

from mundo import *
```

Localización de los módulos

Al importar un módulo Python lo buscara en los siguientes directorios:

1. En el directorio actual.
2. En los directorios declarados en el `PYTHONPATH` (variable de entorno que contiene un listado de directorios)
3. En el directorio de instalación de Python por defecto (en UNIX normalmente `'/usr/local/lib/python/'`)

Paquetes

Es posible agrupar los módulos que tienen relación en un mismo directorio. Estos directorios son conocidos en Python como paquetes y deben contener siempre un archivo llamado `__init__.py` para que Python lo reconozca como un paquete.

A medida que desarrollamos una aplicación es habitual agrupar los archivos en directorios (paquetes) para tener el código organizado.

Para cargar un módulo ubicado en un paquete lo haremos de la siguiente forma:

```
import mipaquete.mundo
```

o bien de la siguiente manera:

```
from mipaquete import mundo
```

También es posible importar elementos concretos de un módulo:

```
from mipaquete.mundo import adios_mundo, hola_mundo
```